

燕山大学

《微型计算机原理》平时作业

定时器中断下的 2-10 进制转换器

学院（系）： 电气工程学院

年级专业： 22 级检测 2 班

学生姓名： 游寅龙

学 号： 202211030001

完成日期： 2024 年 6 月 8 日

摘要

本研究旨在使用 8086 芯片作为主控芯片控制多个外设模块，实现嵌入式系统中的信息获取、处理、显示等功能。

通过编程配置并行接口芯片 8255、中断处理芯片 8259、可编程计数器芯片 8253 以将按键数据获取后传输至数码管模块和点阵模块。

实验结果表明，使用 8255 操作点阵模块可以有效实现信息显示，为嵌入式系统的交互性和可视化展示提供了重要支持。

关键词：8255 芯片、8259 芯片、8253 芯片、点阵模块、汇编语言。

1、 研究背景及意义

1.1 研究背景

定时器中断是微型计算机中实现精确时间计时的关键。

定时器提供了一种有效的机制，使得微机系统能够周期性地触发中断服务函数，从而实现实时任务调度。通过合理设置定时器的计时周期，系统可以按照预定的时间间隔执行特定的任务，这对于确保系统的高效性和实时性至关重要。

每当定时器中断发生时，计数器的值会自增，从而实现精确的时钟计时功能。这对于需要高精度时间管理的应用场景，如实时控制系统、数据采集与处理等。

因此需要通过软硬件结合的手段实现定时器中断，能够有效节约计算资源，为复杂程序奠定坚实的基础。

同时，点阵和数码管是常用的显示设备，通常使用扫描的方法来显示，但这对于大量的显示需求和较低主频的主控芯片并不适用，因此本文针对此情况进行深入探索。

1.2 研究意义

本文的研究意义主要体现在以下几个方面：

1. 提出使用串行通信的寄存器芯片 MAX7219 和 74LS595 以实现点阵和数码管的操作简化。
2. 尝试使用 NE555 芯片搭建时基电路，经过 8253 分频后作为定时器中断信号给与 8259，通过外部电路实现了 8086 微机的定时器中断操作。
3. 构建了多点阵的阵列显示相关算法。

2、 基本原理

2.1 脉冲发生芯片 NE555

NE555 是一款经典的集成电路，其内部结构 [1] 如图 2-1，包括比较器、RS 触发器、电压比较器和输出级三个主要功能模块，外部引脚则提供了与其他电路元件进行连接的接口。NE555 的设计目的是为了提供一种简单方便的定时器解决方案，它广泛应用于模拟和数字电路中。

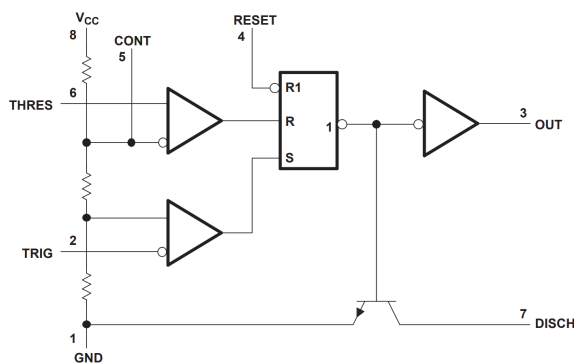


图 2-1: NE555 内部结构框图

NE555 有三种常见的工作模式：单稳态、多稳态和无稳态模式。

在本次仿真实验中需要使用 NE555 搭建脉冲发生电路，因此使用的是其多稳态工作模式，其电路连接方式如图 2-2：

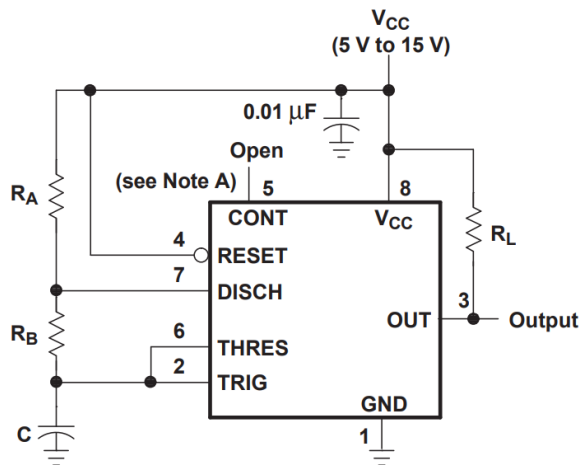


图 2-2: NE555 脉冲发生电路

此电路可以生成稳定的 PWM 方波信号，以下为该信号频率 f 和占空比 $D\%$ 计算公式：

$$f = \frac{1.44}{(R_A + 2R_B)C} \quad (1)$$

$$D\% = \frac{R_A + R_B}{R_A + 2R_B} \times 100\% \quad (2)$$

2.2 可编程定时/计数器 8253

8253 是 Intel 公司生产的三通道 16b 的可编程定时/计数器，是具有 24 根引脚的双列直插式器件。[2]

其内部结构框图 [3] 如图 2-3所示，主要包括三个计数器、一个控制寄存器以及数据总线缓冲器和读写逻辑电路。

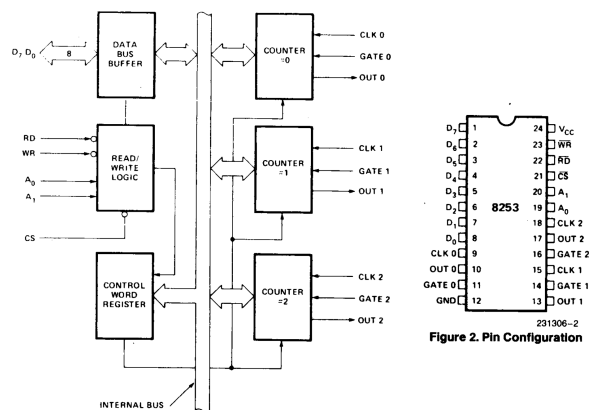


图 2-3: 8253 内部结构框图

其控制字格式和初始化编程顺序如图 2-4所示。本次仿真采用两个分频器串联进行，因此仅使用计数器

Control Word Format

D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	D ₀
SC1	SC0	RL1	RL0	M2	M1	M0	BCD

(a) 8253 控制字格式

		A1	A0
No. 1	MODE Control Word Counter 0	1	1
No. 2	MODE Control Word Counter 1	1	1
No. 3	MODE Control Word Counter 2	1	1
No. 4	LSB Count Register Byte Counter 1	0	1
No. 5	MSB Count Register Byte Counter 1	0	1
No. 6	LSB Count Register Byte Counter 2	1	0
No. 7	MSB Count Register Byte Counter 2	1	0
No. 8	LSB Count Register Byte Counter 0	0	0
No. 9	MSB Count Register Byte Counter 0	0	0

(b) 8253 初始化编程顺序

图 2-4: 8253 初始化

0 和计数器 1，选择两者的工作方式均为方波发生器，只读写计数器低八位。

需要对 2-4b中的初始化顺序进行删减，删去初始化计数器 2 的流程和对计数器高八位（MSB）的写入流程。

2.3 可编程中断控制器 8259A

8259A 是 Intel 公司生产的专为 8086/8088CPU 配套的可编程中断控制器，用于对 8086/8088 系统中的可屏蔽中断进行管理。8259A 可对 8 个中断源实现优先级控制。8259A 可以根据不同的中断源向 CPU 提供不同的中断类型码，还可根据需要对中断源进行中断屏蔽。8259A 有多种工作方式，可以通过编程来选择，以适应不同的应用场合。[2]

其内部结构框图 [4] 如图 2-5所示。

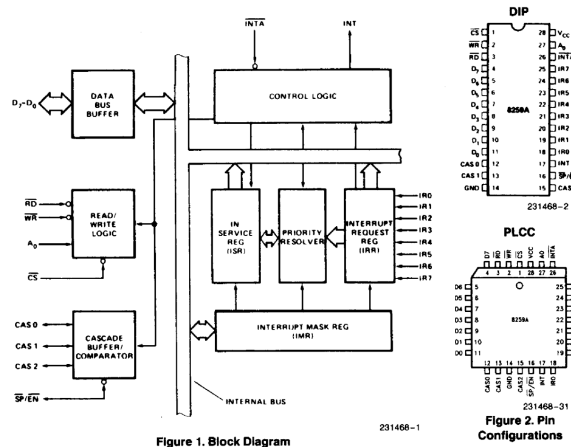


图 2-5: 8259A 内部结构框图

其初始化涉及四条初始化命令字（ICW）和三条操作命令字（OCW），由于篇幅限制将不在此列出具体指令的含义。

其控制字格式和初始化编程顺序如图 2-6所示。本次仿真只有一个中断源。因此，不使用初始化命令字

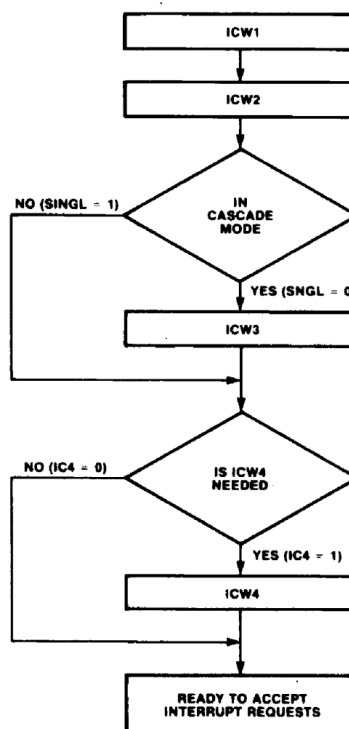


图 2-6: 8259A 初始化编程顺序

ICW3 和操作命令字 OCW1、OCW3。

对于 8086 芯片，中断需要向内存中的中断向量表中指明中断服务函数的代码段基地址和子程序偏移地址，故在初始化 8259A 后应当初始化向量表。

2.4 可编程并行接口 8255

Intel 8255A 是一个通用目的可编程 I/O 设备，设计用于与 Intel 微处理器一起使用。它拥有 24 个 I/O 引脚，这些引脚可以单独编程为两组各 12 个引脚，并在三种主要操作模式下使用。在第一种模式（模式 0）中，每组 12 个 I/O 引脚可以编程为每组 4 个引脚作为输入或输出。在第二种模式（模式 1）中，每组可以编程为具有 8 条输入或输出线。在剩余的 4 个引脚中，有 3 个用于握手和中断控制信号。第三种操作模式（模式 2）是双向总线模式，它使用 8 条线作为双向总线，并借用另一组的 1 条线用于握手，共 5 条线。[5] 其内部结构框图 [5] 如图 2-7 所示。

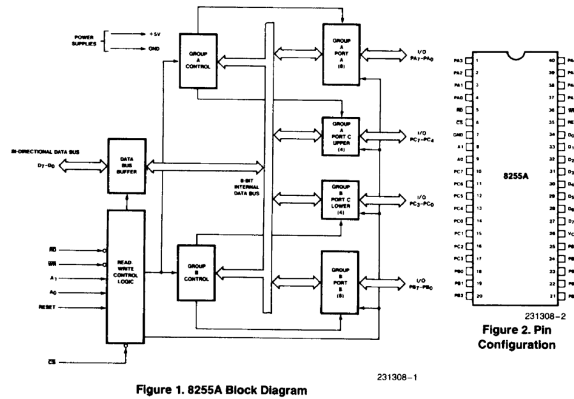


图 2-7: 8255 内部结构框图

本次仿真使用 8255 的并行口均在模式 0 下工作，其中，PA 口作为输出，PB 口作为输入，根据 8255 芯片控制字规则可知道对应的控制字应为 10000010B=82H。

2.5 LED 显示芯片 MAX7219

MAX7219/MAX7221 为紧凑的串行输入/输出共阴极显示驱动器，用于连接微处理器 (μP) 与 8 位 7 段 LED 数码管显示器、条形图显示器或 64 个独立的 LED。器件内置 BCD B 码译码器、多路复用扫描电路、段和位驱动器以及存储每位数字的 8x8 静态 RAM。只需一个外部电阻即可设置所有 LED 的段电流。MAX7221 兼容于 SPI™、QSPI™ 以及 MICROWIRE™ 接口，段驱动器带有摆率限制，以降低 EMI。[6]

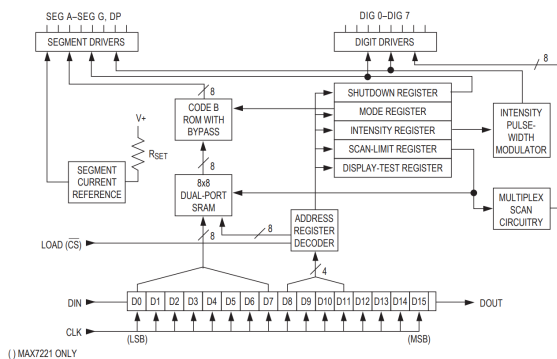


Table 2. Register Address Map

REGISTER	D15-D12	D11	D10	D9	D8	HEX CODE
No-Op	X	0	0	0	0	0x00
Digit 0	X	0	0	0	1	0x01
Digit 1	X	0	0	1	0	0x02
Digit 2	X	0	0	1	1	0x03
Digit 3	X	0	1	0	0	0x04
Digit 4	X	0	1	0	1	0x05
Digit 5	X	0	1	1	0	0x06
Digit 6	X	0	1	1	1	0x07
Digit 7	X	1	0	0	0	0x08
Decode Mode	X	1	0	0	1	0x09
Intensity	X	1	0	1	0	0x0A
Scan Limit	X	1	0	1	1	0x0B
Shutdown	X	1	1	0	0	0x0C
Display Test	X	1	1	1	1	0x0F

(b) MAX7219 寄存器地址图

图 2-8: MAX7219 芯片参数图

MAX7219 内部结构框图如图 2-8a 所示，其内部自带多路复用扫描电路（MULTIPLEX SCAN CIR-

CUITRY), 可以自动进行扫描显示, 方便了点阵的显示。

接口使用 SPI 时序, 高八位地址, 低八位数据的格式通信, 其具体寄存器组如图 2-8b 所示, 寄存器的使用将在软件编程初始化中体现。

2.6 移位寄存器 74LS595

74LS595 是串行输入并行输出的器件, 可以串联使用, 达到引脚拓展的目的。主要引用在数码管的驱动, 点阵的驱动等需要多个引脚的地方。

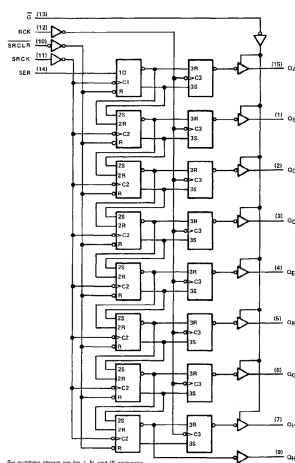


图 2-9: 74LS595 内部结构框图

74LS595 芯片内部结构如图 2-9 所示 [7], 其本质是八个主从 D 触发器串联, 其八位输出每个分别与一个三态门串联。

其中, 输入时序与 SPI 时序类似, 只是多了个 SCK 的输出使能引脚。可以参考软件 SPI 时序的写法结合可编程并行接口芯片 8255 的特点进行代码编写。

3、 硬件设计

硬件整体设计分为如下四个部分：

1. 定时器中断部分硬件设计；
2. 点阵显示部分硬件设计；
3. 数码管显示部分硬件设计；
4. 按键输入部分硬件设计。

根据上述任务简述，绘制出硬件电路总图，如图 3-1。

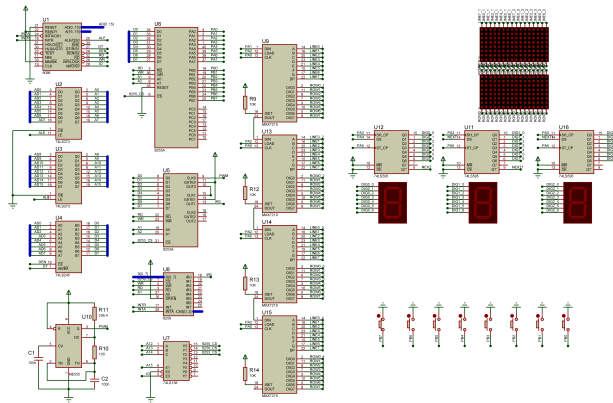


图 3-1: 硬件设计总图

如下，将对每个部分的设计一一作出介绍。

3.1 定时器中断部分

定时器部分主要采用脉冲发生器 NE555 芯片、可编程定时/计数器芯片 8253 和可编程中断控制器芯片 8259A 完成，其原理图如图 3-2所示：

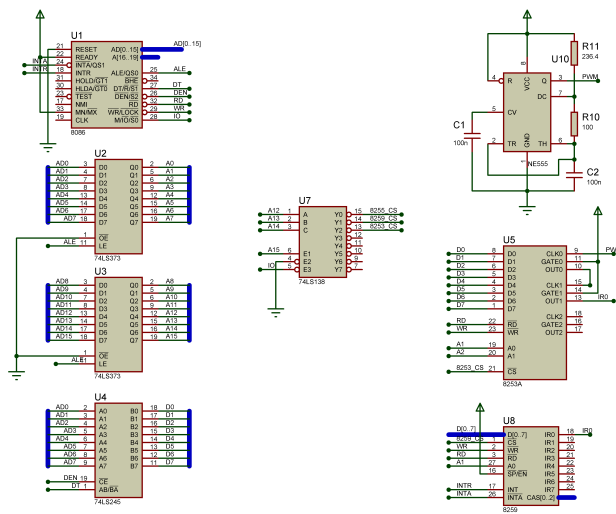


图 3-2: 定时器中断部分原理图

电路采用 NE555 芯片作为时基信号发生器，生成 32768Hz 的频率，根据公式 1，可反推出对应电

阻电容计算公式，因为此处信号只对周期时间有所要求，对占空比无要求，故可采用值较多，此处选择了 $R_A = 236.4\Omega$ 、 $R_B = 100\Omega$ 、 $C = 100nF$ 的组合。

生成的信号将输入 8253 的计数器 0，经过计数器 0 的分频后再次进入计数器 1。通过两次的计时器分频可以将输入 32768Hz，十五倍分频得到 1Hz 的标准秒频率。以此达到每一秒触发一次定时器中断的目的。

其中，8253 的 A0 和 A1 引脚分别与 8086 芯片的 AD1 和 AD2 相通，片选引脚 $\overline{CS}$ 与译码器芯片的 $\overline{Y2}$ ，故可以推得对应其映射地址，如表 3-1 所示。

表 3-1: 8253 芯片的外设地址

端口	二进制地址	采用值
计数器 0	1011 xxxx xxxx x00xB	A000H
计数器 1	1011 xxxx xxxx x01xB	A002H
计数器 2	1011 xxxx xxxx x10xB	A004H
控制字	1011 xxxx xxxx x11xB	A006H

最后将 8253 的输出信号与 8259A 芯片的 IR0 相连，以此将中断信号传入中断控制器芯片，中断控制器的 INT 引脚与 8086 的 $\overline{INTA}/QS1$ 引脚、 $\overline{INTR}$ 引脚 8086 的 INTR 引脚相连，达到将可屏蔽中断信号传入 CPU 中的目的。从而达到引发中断。

其中，8259A 的 A0 引脚与 8086 芯片的 AD1 相通，片选引脚 $\overline{CS}$ 与译码器芯片的 $\overline{Y1}$ ，故可以推得对应其映射地址，如表 3-2 所示。

表 3-2: 8259A 芯片的外设地址

控制字	二进制地址	采用值
ICW1	1010 xxxx xxxx xx0xB	9000H
ICW2	1010 xxxx xxxx xx1xB	9002H
ICW3	1010 xxxx xxxx xx1xB	9002H
ICW4	1010 xxxx xxxx xx1xB	9002H
OCW1	1010 xxxx xxxx xx1xB	9002H
OCW2	1010 xxxx xxxx xx0xB	9000H
OCW3	1010 xxxx xxxx xx0xB	9000H

以上，经过信号的发生，处理，最终以中断信号的形式输入到 8086CPU 中完成定时器中断的信号逻辑。

3.2 点阵显示部分

点阵显示部分采用 LED 显示芯片 MAX7219 驱动，8255 作为预驱，其原理图如图 3-3 所示。

该部分，信号从 8086 输出后，经过 8255 的 PA 并行接口的引脚 PA0、PA1、PA2 三根信号线输出给 MAX7219，MAX7219 接收端从图 2-8a 中可以看出是一个 16 比特的移位寄存器，从 Din 引脚输入，至 Dout 引脚输出，单独芯片完成 16 位数据收发

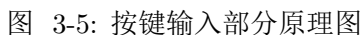
此处，特意设置 CLK 引脚连接 PA0，Din 引脚连接 PA1。如此可以实现，当输入数据的一刻，时钟线是低电平，下一个时刻通过 INC 将时钟拉高，如此减少了代码量。

需要注意的是 MAX7219 芯片 DIGx 引脚原本应连接共阴极数码管的共阴极 COM 端，在 8x8 的数码管中，则应连接行选线。以及本次仿真使用 4 个 8*8 点阵构成，所以需要级联四片 MAX7219 芯片以驱动，每两片芯片之间 Din 连下一级的 Dout。

仿真采用 7 位共阴极数码管，对于小数点 dp 位并没有固定要求，其七段码对应如表 3-4，本芯片与 MAX7219 共用 8255 芯片，因此地址保持不变，详见表 3-3 所示。

数字	二进制七段码	采用值
0	1111 110xB	0FCH
1	0110 000xB	060H
2	1101 101xB	0DAH
3	1111 001xB	0F2H
4	0110 011xB	066H
5	1011 011xB	0B6H
6	1011 111xB	0BEH
7	1110 000xB	0E0H
8	1111 111xB	0FEH
9	1111 011xB	0F6H

按键部分采用 8255 作为驱动, 其原理图如图 3-5 所示。



同时, 按键部分与 MAX7219 共用 8255 芯片, 因此地址保持不变, 详见表 3-3 所示。

4、 软件设计

整体流程图如图 4-1所示。

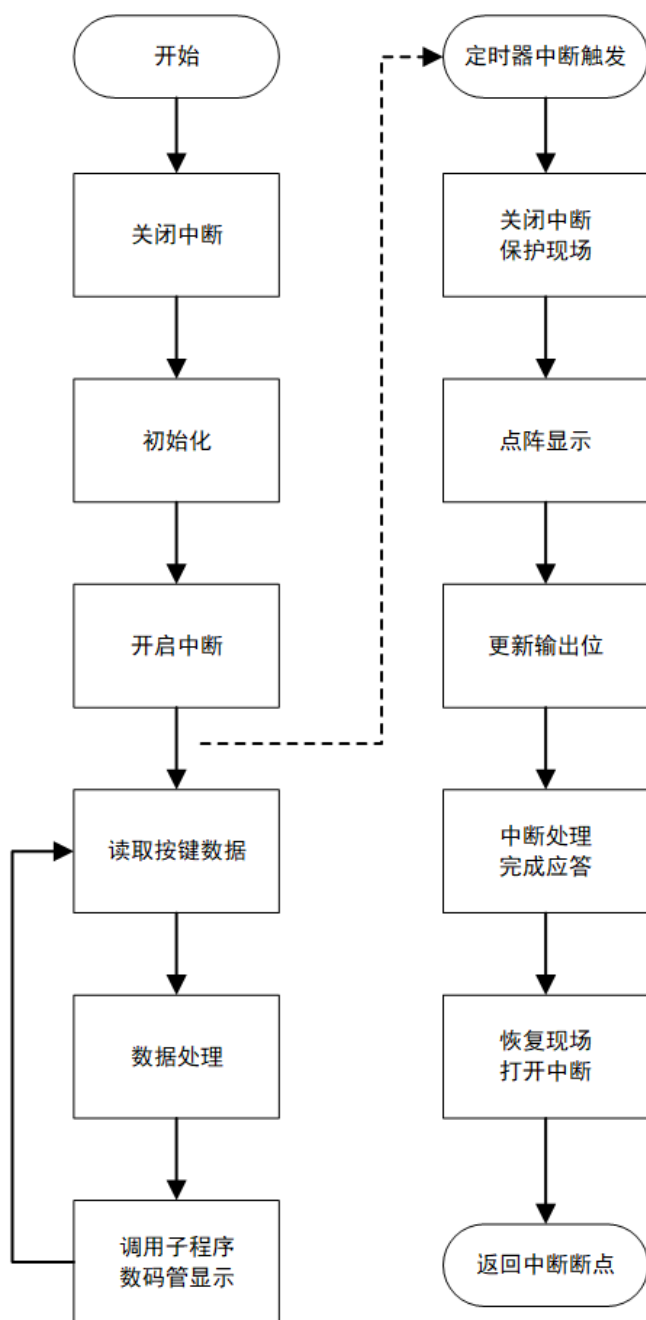


图 4-1: 总体流程图

整体流程大致可分为三个部分：

1. 各芯片初始化部分；
2. 主循环部分；
3. 定时器中断部分。

首先，8086 会屏蔽中断，执行各芯片初始化部分，初始化后打开中断，若中断触发，则执行定时器中断部分的点阵显示中文循环，否则执行主循环部分，读取按键后，将二进制译作十进制后再数码管中显示。

如下，将对每个部分的设计一一作出详细介绍。

4.1 各芯片初始化部分

各芯片初始化部分需要对 5 块芯片以及中断向量表初始化，具体流程如下图 4-2 所示。

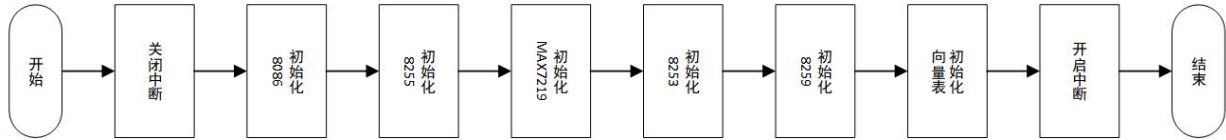


图 4-2: 初始化流程图

第一步关闭中断，指使用 CLI 指令屏蔽可屏蔽中断。

第二步初始化 8086，指为 8086 芯片的数据段寄存器（DS），堆栈段寄存器（SS）和附加段寄存器（ES）赋初值。

第三步初始化 8255，选择 8255 的控制字地址（可查表 3-3）：8006H，为该寄存器复制 82H，表明 A 口工作模式 0，输出；B 口工作模式 0，输入；C 口均作为输出。

第四步初始化 MAX7219，调用 MAX7219 初始化子程序完成，该函数针对图 2-8b 中给出的地址为 0xX9-0xXC，0xFF 的寄存器分别赋值。由于四片串联，且为了保证四个点阵显示条件相同，因此直接连续四次写入相同的地址和数据后拉起寄存器输入信号引脚。方便初始化，也便于代码编写。具体寄存器作用在该芯片的数据芯片中已经给出，使用含义在附录中的代码后也以注释形式注明。

第五步初始化 8253，按照图 2-4b 中给出的初始化顺序，根据表 3-1 中给出的外设地址，先向控制字地址 A006H，先后申请打开计数器 0 和计数器 1，随后向计数器 0 地址 A000H 和计数器 1 地址 A002H 写入对应分频系数。但此处需要说明的是，由于仿真软件 Proteus 的时间流速和客观时间的流速是不一致的，因此为了快速得到相关的实验现象需要将分频系数降低，也就是定时器中断时间简短。基于仿真环境的时间流速和芯片最高传输速率的考量，最终选择计数器 0 的计数值为 6FH，计数器 1 的计数值为 4FH。

第六步初始化 8259，按照图 2-6 中给出的初始化顺序和硬件图，判断出不必写入级联控制字 ICW3，随后按照表 3-2 中给出的控制字地址，向初始化字 ICW1 中写入单片中断以及上升沿触发，中断向量码 ICW2 中写入中断向量码为 60H-67H，跳过级联控制字 ICW3 后，向中断方式结束字 ICW4 中写入一般嵌套、非缓冲模式。由于八条中断线只使用一条，因此还需要提交操作命令字 OCW1，频率高六位中断。

第七步初始化中断向量表，首先建立中断服务函数 INT0，随后根据向量表建立在段基地址为 0H 的规则，向附加段寄存器（ES）中赋值 0H，以方便向量表的写入。根据第六步中中断向量码 ICW2 中所写起始地址 60H，和硬件中断信号连入 IR0，计算得到对应中断服务函数的 IP 地址应当写入首地址为 60H×4H 的字中，CS 地址应当写入首地址为 60H×4+2H 的字中。

第八步打开中断，指使用 STI 指令开启可屏蔽中断。

以上八个步骤完成了整体系统的初始化，接下来将在定时器中断信号中进行主函数的循环。

4.2 主循环部分

主循环部分功能主要是读取按键，将其对应 2-10 进制转换后输出至数码管即可。具体流程如下图 4-3 所示。

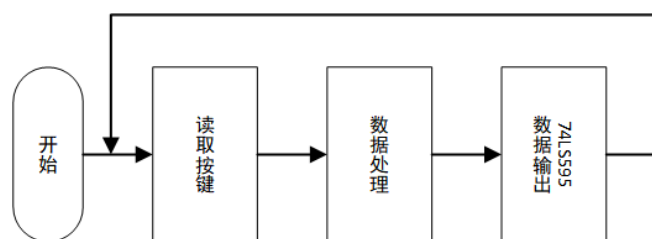


图 4-3: 主循环流程图

第一步，读取按键输入，该步只需要读取对应的 PB 口输入即可，PB 寄存器地址已在表 3-3 中给出。

第二步，数据处理，由于采用的是上拉电阻的方案，因此得到的数据采用反逻辑编码，与人们所熟知的正逻辑相悖，因此需要使用 NOT 指令转换逻辑。

第三步，调用 DIGITAL_OUTPUT 子程序输出。该子程序将需要处理的数据按十进制分为个位、十位、百位。查表 3-4，得到对应七段码，按低位优先的顺序，逐个推入 74LS595 芯片中。

完成以上三步后，进行无条件跳转，用以实现主函数的死循环。

4.3 定时器中断部分

定时器中断部分功能主要是控制点阵输出，并计算下一次中断点阵显示字的序号。具体流程如下图 4-4 所示。



图 4-4: 定时器中断流程图

第一步，关闭中断，使用 CLI 指令屏蔽可屏蔽中断。

第二步，保护现场，由于中断的指令将会充分使用四个数据寄存器，且中断时刻无法确定主函数正在执行哪条语句，因此，需要将四个数据寄存器的内容都分别推入栈中，加以保护。

第三步，点阵输出，首先需要从数据段中找出所要输出汉字的汉字码信息，并且由于数据量较大，因此应当选用基址变址相对寻址方式进行，但可惜本文由于时间关系并没有很好的做到这个方案，而是先将变址与相对地址相加，在和基址一并查表得出，实现了既定目标，但是代码较为杂乱。接着是点阵输出的主要方法是行选，因为 loop 指令是 CX 截止到 0H 时停止循环运行，因此，此处从高位行往低位行扫描是一个较为良好的方案。每次推送四个点阵的最低位数据随后拉起信号线。以此重复八次后，得到了一个完整字的全部输出，也就完成了点阵输出。

第四步，计算下次中断输出，其本质是对计数器 CHAR_NUMBER 进行加 1 操作，但是需要注意的是，需要对最后一位跳转到第一位这个边界条件进行一定的条件运算处理。本文给出直接与总字数作比的方案，随后考察 CF 标志位的方案。

第五步，告知中断芯片处理结束，使得下次中断得以正常接收。本质是写中断结束操作命令字 OCW2，向其写入中断结束指令。

第六步，恢复现场，将保护现场中的内容取出，使得主函数的数据恢复。

第七步，打开中断，使用 STI 指令开启可屏蔽中断。

以上七步构成完整的定时器中断。

5、 现象演示

5.1 点阵现象

点阵由定时器中断控制，循环显示“检测二班游寅龙”七个字，如图 5-1。

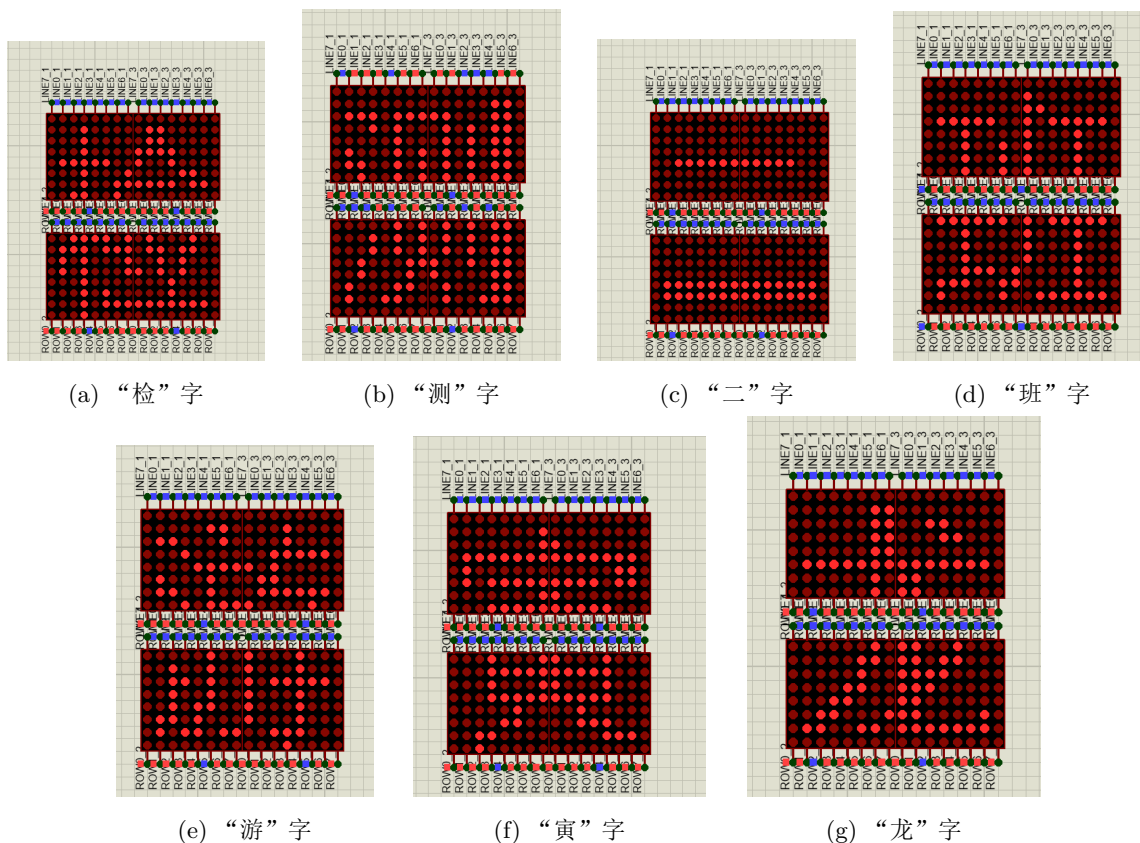


图 5-1: 点阵七位汉字显示

此现象证明定时器中断部分代码正确，同时也说明了代码为本人所著。

5.2 数码管现象

数码管数字为按键的二进制转换，按人们习惯，按键按下记为 1，按键断开记为 0。以下试举两例，如图 5-2。

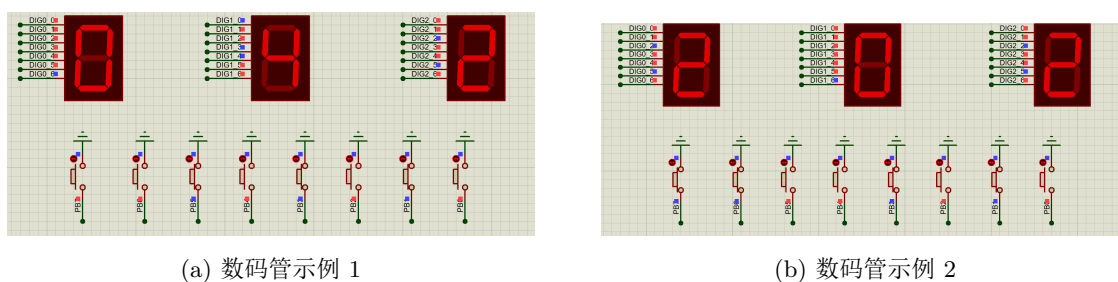


图 5-2: 数码管译码举例示范

对比按键所示二进制和数码管显示十进制，两者表达同一数字，证明主函数部分代码正确。

6、 总结

自从我深入探索微机原理的世界，我对计算机科学的理解达到了一个新的高度。这不仅仅是一门学科的学习，更是一次自我挑战和成长的旅程。

随着课程的深入，我逐渐感受到了计算机科学的魅力。从最初的 CPU、内存、I/O 设备等基础概念，到后来的指令系统、存储系统、输入输出系统等复杂知识，每一步的学习都让我对计算机有了更深刻的认识。

在学习 CPU 的内部结构和工作流程时，我被其设计的精妙所震撼。它像一个高效的工厂，通过执行各种指令来完成复杂的计算任务。这让我深刻体会到，计算机科学不仅仅是理论的学习，更是对实际应用的深入理解和探索。

在编写汇编程序的过程中，我亲身体验了从高级语言到机器语言的转换过程。这一过程让我对计算机程序的工作原理有了更直观的认识。同时，我也意识到编程不仅仅是一种技能，更是一种思维方式的训练。它要求我们具备严密的逻辑思维和解决问题的能力。

在本次的大作业中，我也亲手结合硬件和软件，搭建了 8086 处理系统，虽然因为时间的原因，这个系统不管是从硬件角度还是软件都有很多的提升改进之处，但是这次大作业让我深刻对微机的工作方式有了理解，对于芯片之间的连接控制有了更系统的了解，也扩展了我的眼界。

总的来说，微机原理的学习让我对计算机科学有了更深入的理解。我不仅掌握了相关的知识和技能，还培养了严密的逻辑思维和解决问题的能力。我相信，在未来的学习和工作中，我将继续深入探索微机原理的奥秘，为计算机科学的发展贡献自己的力量。同时，我也希望能够将这些知识应用到实际中，为社会的发展做出自己的贡献。

参考文献

- [1] xx555 Precision Timers, 3rd ed., Texas Instruments, 9 2014. [Online]. Available: www.ti.com
- [2] 吴宁, 微型计算机原理与接口技术, 4th ed. 清华大学出版社, 2 2002.
- [3] 8253/8253-5 PROGRAMMABLE INTERVALTIMER, Intel, 11 1986.
- [4] 8259A PROGRAMMABLE INTERRUPT CONTROLLER, Intel, 12 1986.
- [5] 8255A/8255A-5 PROGRAMMABLE PERIPHERAL INTERFACE, Intel, 8 1991.
- [6] MAX7219/MAX7221 Serially Interfaced, 8-Digit LED Display Drivers, 6th ed., maxim integrated, Maxim Integrated 160 Rio Robles, San Jose, 8 2003.
- [7] 8-BIT SHIFT REGISTERS WITH OUTPUT LATCHES, Texas Instruments, 3 1988. [Online]. Available: www.ti.com

附 录（程序清单）

```

1  DATASE SEGMENT
2      C8253CTRL EQU 0A006H
3      C8253A EQU 0A000H
4      C8253B EQU 0A002H
5      C8253C EQU 0A004H ;8253 地址
6
7      C8255CTRL EQU 08006H
8      C8255A EQU 08000H
9      C8255B EQU 08002H
10     C8255C EQU 08004H ;8255 地址
11
12     C8259ICW1 EQU 09000H
13     C8259ICW2 EQU 09002H
14     C8259ICW3 EQU 09002H
15     C8259ICW4 EQU 09002H ;8259A 初始化命令字地址
16
17     C8259OCW1 EQU 09002H
18     C8259OCW2 EQU 09000H
19     C8259OCW3 EQU 09000H ;8259A 操作命令字地址
20
21
22     MAX7219_ADDR DB 00H ;待访问的MAX7219寄存器地址（MAX7219_WRITE_DATA调用）
23     MAX7219_DATA DB 00H ;待访问的MAX7219寄存器数据（MAX7219_WRITE_DATA调用）
24     MAX7219_DIN_H EQU 02H ;CS=0 DIN=1 CLK=0（INC DIN_H能使得时钟线拉高）
25     MAX7219_DIN_L EQU 00H ;CS=0 DIN=0 CLK=0（INC DIN_L能使得时钟线拉高）
26     MAX7219_CS_H EQU 04H ;CS=1
27
28     LS595_OUTPUT DB 00H ;待74LS595输出的数据（DIGITAL_OUTPUT调用）
29     LS595_DAT_H EQU 14H ;RCK=0 DAT=1 SCK=0（ADD DAT,08H能使得时钟线拉高）
30     LS595_DAT_L EQU 04H ;RCK=0 DAT=0 SCK=0（ADD DAT,08H能使得时钟线拉高）
31     LS595_RCK_H EQU 24H ;RCK=1
32     LS595_RCK_L EQU 04H ;RCK=0
33
34     CHAR_NUMBER DB 00H ;指示点阵下一个显示的字（INT0调用）
35
36     DIGITAL_NUM ;共阴极数码管字库，每个字节代表一个阿拉伯数字
37     DB 0FCH,060H,0DAH,0F2H,66H,0B6H,0BEH,0E0H,0FEH,0F6H;
38
39     NAME_CHAR ;点阵字库，每两行代表一个字，“检测二班游寅龙”
40     DB 000H,010H,010H,010H,07CH,011H,013H,032H,03DH,077H,051H,051H,010H,014H,017H,010H;
41     DB 000H,060H,060H,0F0H,090H,00CH,0F6H,000H,044H,06CH,028H,0A8H,010H,010H,0FEH,000H;
42     DB 000H,000H,077H,014H,004H,045H,065H,025H,005H,015H,015H,023H,022H,046H,04CH,000H;
43     DB 000H,006H,0C6H,056H,056H,056H,056H,056H,056H,056H,096H,086H,046H,04EH,000H;
44     DB 000H,000H,000H,000H,03FH,000H,000H,000H,000H,000H,000H,07FH,07FH,000H,000H;
45     DB 000H,000H,000H,000H,0F8H,000H,000H,000H,000H,000H,000H,0FEH,0FEH,000H,000H;
46     DB 000H,000H,000H,07CH,010H,012H,012H,012H,07EH,010H,010H,010H,01DH,073H,042H,000H;
47     DB 000H,080H,0C0H,0BEH,088H,088H,088H,088H,0BEH,088H,088H,088H,008H,008H,07EH,000H;
48     DB 000H,006H,062H,010H,00FH,044H,064H,017H,004H,024H,034H,024H,02CH,068H,05BH,000H;

```

```

49         DB 000H,010H,010H,03EH,0A0H,060H,07EH,084H,088H,088H,0BEH,088H,088H,088H,038H,010H;
50         DB 000H,001H,001H,07FH,040H,05FH,001H,01FH,011H,01FH,011H,01FH,004H,00CH,030H,020H;
51         DB 000H,000H,080H,0FEH,006H,0F6H,080H,0F8H,088H,0F8H,088H,0F8H,020H,038H,00EH,000H;
52         DB 000H,003H,003H,003H,07FH,002H,002H,002H,006H,004H,00CH,018H,031H,066H,000H;
53         DB 000H,000H,030H,018H,000H,0FEH,0C0H,0C0H,0C8H,0D8H,0F0H,0E0H,0C0H,0C2H,0FEH,000H;
54 DATASE ENDS
55
56 STACKSE SEGMENT STACK
57         DW 256H DUP(?)
58 STACKSE ENDS
59
60 EXTRASE SEGMENT
61         DW 256H DUP(?)
62 EXTRASE ENDS
63
64 CODESE SEGMENT PUBLIC 'CODE'
65 ;;;;;;;;;; 主函数 ;;;;;;;;;;
66     MAIN PROC
67         ASSUME CS:CODESE, DS:DATASE, SS:STACKSE, ES:EXTRASE
68     START:
69         CLI                                ; 关闭中断
70
71         MOV AX, DATASE
72         MOV DS, AX
73         MOV AX, STACKSE
74         MOV SS, AX
75         MOV AX, EXTRASE
76         MOV ES, AX                        ; 段寄存器赋值
77
78         MOV DX, C8255CTRL
79         MOV AL, 10000010B
80         OUT DX, AL                        ; 8255 初始化——A口输出，B口输入
81
82         CALL MAX7219_INIT                ; 调用子程序初始化MAX7219
83
84         MOV DX, C8253CTRL
85         MOV AL, 00010110B
86         OUT DX, AL                        ; 8253 初始化——打开计数器0
87
88         MOV DX, C8253CTRL
89         MOV AL, 01010110B
90         OUT DX, AL                        ; 8253 初始化——打开计数器1
91
92         MOV DX, C8253A
93         MOV AL, 006FH
94         OUT DX, AL                        ; 8253 初始化——计数器0计数6FH
95
96         MOV DX, C8253B
97         MOV AL, 04FH

```

```

98      OUT DX, AL                ;8253 初始化——计数器0 计数4FH
99
100     MOV DX, C8259ICW1
101     MOV AL, 00010011B
102     OUT DX, AL                ;8259A 初始化——设置为单片上升沿触发
103
104     MOV DX, C8259ICW2
105     MOV AL, 060H
106     OUT DX, AL                ;8259A 初始化——设置IR0—IR7 对应中断向量60H—67H
107
108     MOV DX, C8259ICW4
109     MOV AL, 00000001B
110     OUT DX, AL                ;8259A 初始化——设置为一般嵌套、非缓冲
111
112     MOV DX, C8259OCW1
113     MOV AL, 11111100B
114     OUT DX, AL                ;8259A 初始化——屏蔽高六位中断，只保留IR0、IR1
115
116     MOV AX, 00H
117     MOV ES, AX                ;定义中断向量表——附加段段基地址指向向量表段基地址
118
119     MOV BX, 60H*4
120     MOV AX, OFFSET INTO
121     MOV ES:[BX], AX           ;定义中断向量表——写入中断服务函数的偏移地址
122
123     MOV AX, CS
124     MOV ES:[BX+2], AX         ;定义中断向量表——写入中断服务函数的段基地址
125
126     STI                        ;开启中断
127
128     LOOP_C:
129     MOV DX, C8255B
130     IN AL, DX                 ;读取按键数据
131
132     NOT AL                     ;由于按键采用反逻辑，因此需要改为正逻辑
133
134     MOV LS595_OUTPUT, AL
135     CALL DIGITAL_OUTPUT       ;将二进制交予数码管处理函数处理
136
137     JMP LOOP_C                ;跳回主函数形成死循环，防止停机
138
139     RET                        ;实际运行不到，为了防止编译器给予警告
140     MAIN ENDP
141
142     ;;;;;;;;;;MAX7219 初始化函数;;;;;;;;;
143     MAX7219_INIT PROC
144         MOV MAX7219_ADDR, 09H
145         MOV MAX7219_DATA, 00H ;译码方式设为不译码，地址0xX9，数值0x00
146         CALL MAX7219_WRITE_DATA

```

```

147      CALL MAX7219_WRITE_DATA
148      CALL MAX7219_WRITE_DATA
149      CALL MAX7219_WRITE_DATA ; 四片芯片都是该译码方式
150      MOV DX, C8255A
151      MOV AL, MAX7219_CS_H
152      OUT DX, AL ; 传送结束，拉起片选信号
153
154      MOV MAX7219_ADDR, 0AH ; 亮度为7/32，地址0xA，数值0x3
155      MOV MAX7219_DATA, 03H
156      CALL MAX7219_WRITE_DATA
157      CALL MAX7219_WRITE_DATA
158      CALL MAX7219_WRITE_DATA
159      CALL MAX7219_WRITE_DATA ; 四片芯片都是该亮度
160      MOV DX, C8255A
161      MOV AL, MAX7219_CS_H
162      OUT DX, AL ; 传送结束，拉起片选信号
163
164      MOV MAX7219_ADDR, 0BH
165      MOV MAX7219_DATA, 07H ; 扫描界限为8行，地址0xB，数值0x7
166      CALL MAX7219_WRITE_DATA
167      CALL MAX7219_WRITE_DATA
168      CALL MAX7219_WRITE_DATA
169      CALL MAX7219_WRITE_DATA ; 四片芯片都是该扫描界限
170      MOV DX, C8255A
171      MOV AL, MAX7219_CS_H
172      OUT DX, AL ; 传送结束，拉起片选信号
173
174      MOV MAX7219_ADDR, 0CH
175      MOV MAX7219_DATA, 01H ; 关断寄存器为普通模式，地址0xC，数值0x1
176      CALL MAX7219_WRITE_DATA
177      CALL MAX7219_WRITE_DATA
178      CALL MAX7219_WRITE_DATA
179      CALL MAX7219_WRITE_DATA ; 四片芯片都是该普通模式
180      MOV DX, C8255A
181      MOV AL, MAX7219_CS_H
182      OUT DX, AL ; 传送结束，拉起片选信号
183
184      MOV MAX7219_ADDR, 0FH
185      MOV MAX7219_DATA, 00H ; 显示器寄存器测试模式为关闭，地址0xF，数值0x0
186      CALL MAX7219_WRITE_DATA
187      CALL MAX7219_WRITE_DATA
188      CALL MAX7219_WRITE_DATA
189      CALL MAX7219_WRITE_DATA ; 四片芯片都是该关闭模式
190      MOV DX, C8255A
191      MOV AL, MAX7219_CS_H
192      OUT DX, AL ; 传送结束，拉起片选信号
193
194      RET ; 返回原函数
195      MAX7219_INIT ENDP

```

```

196 ;;;;;;;;;;MAX7219单片数据写入;;;;;;;;;;
197     MAX7219_WRITE_DATA PROC
198         PUSH AX
199         PUSH BX
200         PUSH CX
201         PUSH DX             ;保护现场
202
203         MOV DX,C8255A       ;设置输出端口为8255的A口（DX以下不再变动）
204
205         ;按照芯片输出时序，先传送地址。
206         MOV BL, MAX7219_ADDR ;传送地址入BX寄存器
207         MOV CL, 08H          ;计数器置为8（一个字节八位）
208     ADDR_LOOP:
209         TEST BL, 80H          ;将BL与80H求与，测试BL最高位是否是1
210         JNZ ADDR_EQU_1        ;跳转到传送低电平方案
211         MOV AL, MAX7219_DIN_L
212         JMP ADDR_EQU_0        ;跳出分支结构
213     ADDR_EQU_1:
214         MOV AL, MAX7219_DIN_H
215     ADDR_EQU_0:
216         OUT DX, AL           ;输出数据（此时时钟线为低）
217         INC AL
218         OUT DX, AL           ;拉高时钟线并输出
219         SHL BX, 01H          ;左移，准备输出下一位
220         LOOP ADDR_LOOP       ;循环8次（每次传送一位）
221
222         ;按照芯片输出时序，再传送数据。
223         MOV BL, MAX7219_DATA ;传送数据入BX寄存器
224         MOV CL, 08H          ;计数器置为8（一个字节八位）
225     DATA_LOOP:
226         TEST BL, 80H          ;将BL与80H求与，测试BL最高位是否是1
227         JNZ DATA_EQU_1       ;跳转到传送低电平方案
228         MOV AL, MAX7219_DIN_L
229         JMP DATA_EQU_0       ;跳出分支结构
230     DATA_EQU_1:
231         MOV AL, MAX7219_DIN_H
232     DATA_EQU_0:
233         OUT DX, AL           ;输出数据（此时时钟线为低）
234         INC AL
235         OUT DX, AL           ;拉高时钟线并输出
236         SHL BX, 01H          ;左移，准备输出下一位
237         LOOP DATA_LOOP       ;循环8次（每次传送一位）
238
239         POP DX
240         POP CX
241         POP BX
242         POP AX               ;恢复现场
243         RET                  ;返回原函数
244     MAX7219_WRITE_DATA ENDP

```

```

245 ;;;;;;;;;;74LS595输出三位数;;;;;;;;;;
246     DIGITAL_OUTPUT PROC FAR
247         PUSH AX
248         PUSH BX
249         PUSH CX
250         PUSH DX             ;保护现场
251
252         MOV DX, C8255A      ;设置输出端口为8255的A口（DX以下不再变动）
253
254         ;先输出个位
255         MOV AX, 00H         ;将AH置0
256         MOV AL, LS595_OUTPUT ;取出需要输出的三位数
257         MOV BX, 10D
258         DIV BL              ;除以10（余数在AH，商在AL）
259         PUSH AX             ;入栈（保留商，也就是十位和百位）
260         MOV CL, 8H
261         ROR AX, CL          ;将余数（个位）移动至AL
262         MOV BX, OFFSET DIGITAL_NUM ;取出数码管字库的数组首地址
263         XLAT                ;查表（BX表示表格首地址，AL表示表内位移量）
264         MOV BL, AL ;将数据放入BX寄存器（OUT指令只允许输出AX，判断需在BX中进行）
265     OUPUT1_LOOP:
266         TEST BL, 01H        ;将BL与80H求与，测试BL最高位是否是1
267         JNZ OUPUT1_EQU_1    ;跳转到传送高电平方案
268         MOV AL, LS595_DAT_L
269         JMP OUPUT1_EQU_0     ;跳出分支结构
270     OUPUT1_EQU_1:
271         MOV AL, LS595_DAT_H
272     OUPUT1_EQU_0:
273         OUT DX, AL          ;输出数据（此时时钟线为低）
274         ADD AL, 08H         ;拉高时钟线并输出
275         OUT DX, AL
276         SHR BX, 01H        ;右移，准备输出下一位
277         LOOP OUPUT1_LOOP    ;循环8次（每次传送一位）
278
279         ;随后输出十位
280         POP AX              ;取出上次除以10后的商和余数
281         AND AX, 00FFH      ;清空AH（清空余数，只保留十位和百位的商）
282         MOV BX, 10D
283         DIV BL              ;除以10（余数在AH，商在AL）
284         PUSH AX             ;入栈（保留商，也就是百位）
285         MOV CL, 8H
286         ROR AX, CL          ;将余数（十位）移动至AL
287         MOV BX, OFFSET DIGITAL_NUM ;取出数码管字库的数组首地址
288         XLAT                ;查表（BX表示表格首地址，AL表示表内位移量）
289         MOV BL, AL ;将数据放入BX寄存器（OUT指令只允许输出AX，判断需在BX中进行）
290     OUPUT2_LOOP:
291         TEST BL, 01H        ;将BL与80H求与，测试BL最高位是否是1
292         JNZ OUPUT2_EQU_1    ;跳转到传送高电平方案
293         MOV AL, LS595_DAT_L

```

```

294         JMP OUPUT2_EQU_0           ;跳出分支结构
295 OUPUT2_EQU_1:
296     MOV AL, LS595_DAT_H
297 OUPUT2_EQU_0:
298     OUT DX, AL                     ;输出数据（此时时钟线为低）
299     ADD AL, 08H                     ;拉高时钟线并输出
300     OUT DX, AL
301     SHR BX, 01H                     ;右移，准备输出下一位
302     LOOP OUPUT2_LOOP                ;循环8次（每次传送一位）
303
304     ;最后输出百位
305     POP AX                           ;取出上次除以10后的商和余数
306     MOV BX, OFFSET DIGITAL_NUM      ;取出数码管字库的数组首地址
307     XLAT                             ;查表（BX表示表格首地址，AL表示表内位移量）
308     MOV BL, AL ;将数据放入BX寄存器（OUT指令只允许输出AX，判断需在BX中进行）
309     MOV CL, 8H                       ;计数器置为8（一个字节八位）
310 OUPUT3_LOOP:
311     TEST BL, 01H                     ;将BL与80H求与，测试BL最高位是否是1
312     JNZ OUPUT3_EQU_1                ;跳转到传送高电平方案
313     MOV AL, LS595_DAT_L
314     JMP OUPUT3_EQU_0                ;跳出分支结构
315 OUPUT3_EQU_1:
316     MOV AL, LS595_DAT_H
317 OUPUT3_EQU_0:
318     OUT DX, AL                     ;输出数据（此时时钟线为低）
319     ADD AL, 08H                     ;拉高时钟线并输出
320     OUT DX, AL
321     SHR BX, 01H                     ;右移，准备输出下一位
322     LOOP OUPUT3_LOOP                ;循环8次（每次传送一位）
323
324     MOV AL, LS595_RCK_H
325     OUT DX, AL ;拉高存储时钟线，从移位寄存器转存带存储寄存器（相当于显示输出）
326
327     MOV AL, LS595_RCK_L
328     OUT DX, AL ;拉低存储时钟线，方便下一次输入数据
329
330     POP DX
331     POP CX
332     POP BX
333     POP AX                           ;恢复现场
334
335     RET                             ;返回原函数
336 DIGITAL_OUTPUT ENDP
337
338 ;;;;;;;;;; 定时器中断服务函数 ;;;;;;;;;;
339 INT0 PROC
340     CLI                             ;关闭中断
341
342     PUSH AX

```

```

343     PUSH BX
344     PUSH CX
345     PUSH DX             ; 保护现场
346
347     MOV AL,CHAR_NUMBER           ; 取出需要输出的汉字位
348     MOV BL,32D
349     MUL BL                     ; 乘以32（一个汉字占32字节）
350     MOV BX,OFFSET NAME_CHAR
351     ADD BX,AX                 ; 将点阵字库数组的地址加上汉字偏移
352                               ; 得到对应汉字存储数组的首地址（相当于基址变址相对寻址）
353
354     MOV CX,08H               ; 计数器置为8（每个点阵八行）
355 NEXT_CHAR:
356     MOV MAX7219_ADDR, CL      ; 给与芯片寄存器地址
357
358     MOV AL,CL
359     ADD AL,23D                ; 推定第四个点阵需要显示的行值
360     PUSH AX                   ; 保存AL（方便其他点阵的推定）
361     XLAT                      ; 查表，获取第四个点阵需要输出的值
362     MOV MAX7219_DATA, AL
363     CALL MAX7219_WRITE_DATA ; 将输出值交由函数负责输出
364
365     POP AX
366     SUB AL,8H                 ; 推定第三个点阵需要显示的行值
367     PUSH AX                   ; 保存AL（方便其他点阵的推定）
368     XLAT                      ; 查表，获取第三个点阵需要输出的值
369     MOV MAX7219_DATA, AL
370     CALL MAX7219_WRITE_DATA ; 将输出值交由函数负责输出
371
372     POP AX
373     SUB AL,8H                 ; 推定第二个点阵需要显示的行值
374     PUSH AX                   ; 保存AL（方便其他点阵的推定）
375     XLAT                      ; 查表，获取第二个点阵需要输出的值
376     MOV MAX7219_DATA, AL
377     CALL MAX7219_WRITE_DATA ; 将输出值交由函数负责输出
378
379     POP AX
380     SUB AL,8H                 ; 推定第一个点阵需要显示的行值
381     XLAT                      ; 查表，获取第一个点阵需要输出的值
382     MOV MAX7219_DATA, AL
383     CALL MAX7219_WRITE_DATA ; 将输出值交由函数负责输出
384
385     MOV AL,MAX7219_CS_H
386     OUT DX,AL                 ; 传送结束，拉起片选信号
387
388     LOOP NEXT_CHAR            ; 循环8次（每次四个芯片各传送一行）
389
390     MOV AL,CHAR_NUMBER
391     INC AL                     ; 输出汉字位加一

```

```

392
393      CMP AL,07H
394      JC CHAR_NUMBER_ISNOT_7 ; 未超出汉字存储位数，跳出分支结构
395      MOV AX,00H ; 超出汉字存储位数，回到第一个汉字
396  CHAR_NUMBER_ISNOT_7:
397      MOV CHAR_NUMBER,AL ; 保存下一次输出汉字位
398
399      MOV DX,C8259OCW2
400      MOV AL,20H
401      OUT DX,AL ; 告知8259A中断处理结束
402
403      POP DX
404      POP CX
405      POP BX
406      POP AX ; 恢复现场
407
408      STI ; 打开中断
409
410      IRET ; 返回中断点
411  INTO ENDP
412 CODESEG ENDS
413      END MAIN

```